B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 6: CONCEPTUAL QUESTION BANK

Virtual Memory, Translation and Page Replacement

Unit	Unit 6
Question type	Conceptual, analytical and scenario-based

Syllabus coverage: Translation Lookaside Buffer (TLB) and Effective Memory Access Time (EMAT); virtual memory, demand paging, swapping and protection; valid-invalid and dirty bits; page-fault handling and free-frame lists; FIFO, LRU and OPT page replacement; thrashing and the working-set model.

A. Translation Lookaside Buffer (TLB) and Effective Memory Access Time

1. What is a Translation Lookaside Buffer (TLB), and why is it used in a paging-based memory-management system?
2. Explain how a TLB hit and a TLB miss differ in the address-translation process.
3. Why can a TLB substantially reduce the average cost of virtual-to-physical address translation?
4. Explain Effective Memory Access Time (EMAT) and identify the main factors that influence it.
5. How do TLB hit ratio, memory access latency and page-table lookup cost affect EMAT?
6. Why can a small decrease in the TLB hit ratio noticeably affect performance when a page-table lookup requires additional memory accesses?
7. What trade-offs arise when increasing TLB size or associativity, including hardware cost, lookup time and hit rate?
8. How can context switches affect TLB contents, and what are the performance implications of flushing or retaining entries?
9. What is the purpose of an Address Space Identifier (ASID) or similar tagging mechanism in a TLB?
10. Why does a TLB miss not necessarily mean that the referenced page is absent from physical memory?
11. How do page faults differ from TLB misses in terms of cause, handling cost and impact on execution?
12. A workload has a high TLB miss rate but few page faults. Explain what this suggests about translation behaviour and what design factors could be investigated.

B. Virtual Memory, Demand Paging, Swapping and Protection

13. Explain virtual memory and how it allows a process to use an address space larger than the physical memory currently assigned to it.
14. How does virtual memory separate a process's logical address space from its current physical-memory residency?
15. Explain demand paging and distinguish it from loading all required pages before a process begins execution.
16. What advantages does demand paging offer in startup time, memory utilization and multiprogramming?
17. What overheads can demand paging introduce, and why can frequent page faults severely degrade performance?
18. Explain swapping and distinguish traditional process swapping from paging individual virtual-memory pages.
19. How can swapping help manage memory pressure, and why can excessive swapping reduce system throughput?
20. How does virtual memory support process isolation while permitting controlled sharing of selected pages?
21. Why must page-level protection be checked during memory access, and what kinds of access violations can it prevent?
22. Discuss how copy-on-write can reduce unnecessary page copying when processes initially share memory.

23. Why is virtual memory not equivalent to having unlimited physical memory, even when the virtual address space is very large?

C. Valid-Invalid and Dirty Bits; Page-Fault Handling; Free-Frame Lists

24. What information can a valid-invalid bit convey in a page-table entry, and how does it help identify illegal or nonresident references?
25. Why should an invalid page reference be distinguished from a valid page that is simply not resident in physical memory?
26. What is a dirty bit, and how does it help the operating system decide whether an evicted page must be written back to storage?
27. Explain the main stages of page-fault handling, from detecting the fault to restarting the interrupted instruction.
28. Why must the operating system verify whether a page fault represents a legal reference before loading the page?
29. What happens when a page fault occurs but no free physical frame is available?
30. Explain the role of a free-frame list and how it helps the operating system allocate frames to incoming pages.
31. Why might a system zero-fill a frame before assigning it to a process, and how does this support security?
32. How can page-fault handling affect process state, scheduling and the execution of other ready processes?
33. Why can dirty-page write-back increase page-replacement latency compared with evicting a clean page?
34. A process repeatedly faults on pages that are valid but not resident. Explain how this differs from an invalid-address protection fault.

D. Page-Replacement Algorithms: FIFO, LRU and OPT

35. Why is page replacement required when a page fault occurs and no free frame is available?
36. Explain the FIFO page-replacement algorithm and discuss its principal advantage and limitation.
37. What is Belady's anomaly, and why is FIFO particularly associated with this phenomenon?
38. Explain Least Recently Used (LRU) page replacement and the locality principle on which it is based.
39. Why can LRU perform well for workloads with temporal locality, and what implementation costs may it introduce?
40. Explain the Optimal (OPT) page-replacement algorithm and why it is useful as a benchmark.
41. Why is OPT generally not implementable as an exact online policy for arbitrary future workloads?
42. Compare FIFO, LRU and OPT in terms of replacement decisions, implementation complexity and practical applicability.
43. Why does the number of page faults alone not fully capture the cost of a page-replacement policy?
44. How can dirty-page write-back, replacement metadata and workload locality affect the practical performance of an algorithm?
45. Why can an algorithm that performs well for one page-reference pattern perform poorly for another?
46. What is the difference between a stack algorithm and a non-stack algorithm, and how does this distinction relate to Belady's anomaly?

E. Thrashing and the Working-Set Model

47. Define thrashing and explain why a system can spend more time servicing page faults than executing useful instructions.
48. How can increasing the degree of multiprogramming contribute to thrashing?
49. Explain the relationship between a process's locality of reference and its demand for resident pages.
50. What is the working-set model, and how does it estimate the pages a process is actively using?
51. How can the working-set model help an operating system decide whether to allocate more frames to a process?

52. Explain how the sum of the working-set demands of active processes can indicate whether physical memory is overcommitted.
53. Why can reducing the degree of multiprogramming help recover from thrashing?
54. Compare working-set-based control with simply increasing the number of frames available to a single process.
55. What trade-offs arise when choosing the working-set window size?
56. How can a system distinguish a temporary burst of page faults from sustained thrashing?
57. A system's CPU utilization falls while disk activity associated with paging rises sharply. Explain why this pattern may indicate thrashing and how the operating system could respond.

F. Integrative and Higher-Order Conceptual Questions

58. A system has a high TLB hit ratio but still experiences poor performance due to frequent page faults. Explain why these observations can coexist.
59. A process has a large virtual address space but actively uses only a small subset of its pages. Explain how demand paging and the working-set model can improve memory utilization.
60. A page-replacement policy evicts a dirty page frequently. Analyze the resulting performance implications and identify workload or policy factors to investigate.
61. Design a conceptual experiment to compare FIFO, LRU and OPT using page-fault counts, memory-write costs and representative workloads.
62. An operating system experiences high page-fault rates after launching several memory-intensive applications. Explain how you would determine whether the cause is insufficient frames, poor locality or thrashing.
63. Explain how TLB design, page-table organization, page-fault handling and page replacement interact to determine effective memory performance.
64. Critically evaluate the statement: 'A high TLB hit ratio guarantees fast memory access.' Include page faults, memory latency and workload behaviour in your reasoning.
65. Recommend a system-level response to sustained thrashing, explaining the trade-offs among frame allocation, admission control and degree of multiprogramming.